



Il software stack nei sistemi HPC

Principi della Programmazione Parallela
CdL Informatica - A.A. 2025/2026



Università
di Catania



INAF
ISTITUTO NAZIONALE
DI ASTROFISICA

Nelle puntate precedenti...

- L'HPC nasce per superare i limiti del calcolo sequenziale
- Le architetture moderne sono:
 - parallele
 - multilivello
 - limitate dall'accesso alla memoria
- Le prestazioni dipendono da:
 - architettura hardware
 - organizzazione dei dati
 - modelli di parallelismo
- I sistemi HPC sono:
 - cluster di nodi multicore
 - con memoria distribuita

Scrivere codice corretto non garantisce buone prestazioni

Obiettivi della lezione

Al termine della lezione lo studente sarà in grado di:

- Comprendere com'è composto un **sistema HPC** dal punto di vista software
 - Capire il ruolo dei principali **componenti dello software stack HPC**
 - Distinguere chiaramente tra:
 - **login node**
 - **compute node**
 - Comprendere perché lo **scheduler** è indispensabile
 - Interpretare il **workflow tipico** di utilizzo di un sistema HPC
 - Essere pronto ad affrontare il **laboratorio**
-

Perché parlare di software stack

Perché un cluster HPC non si usa come un PC o un server tradizionale?

Perché parlare di software stack

Perché un cluster HPC non si usa come un PC o un server tradizionale?

Le risorse dei sistemi HPC sono:

- condivise
- costose
- critiche

Molti utenti competono per:

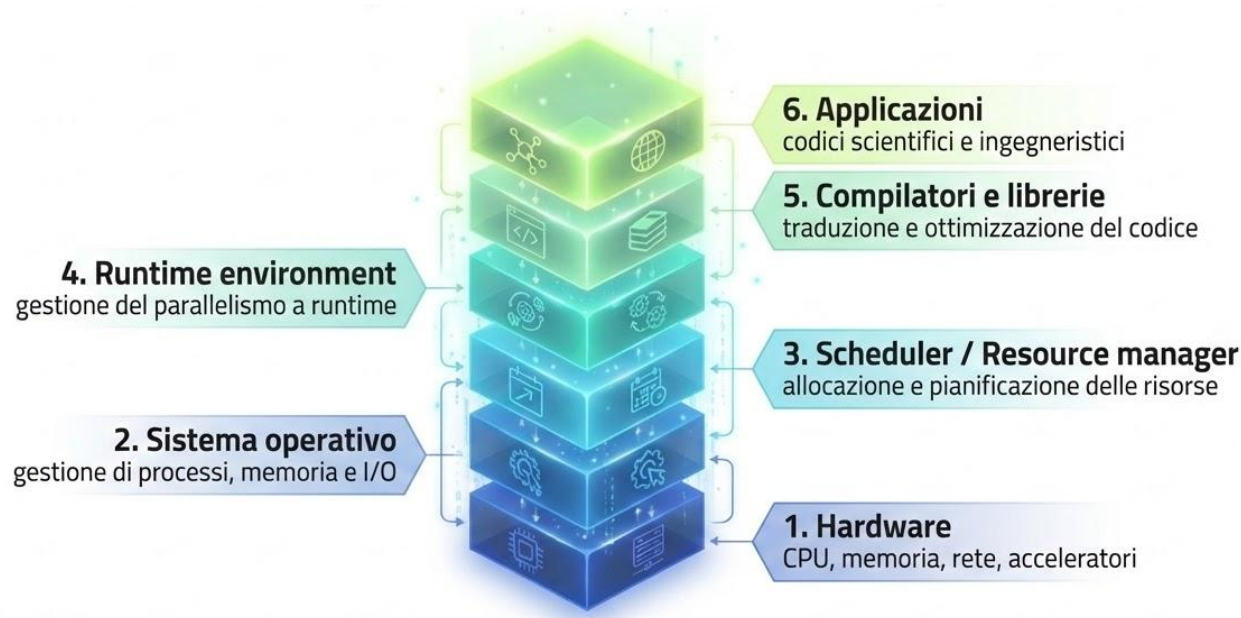
- CPU
- memoria
- rete
- storage

Senza uno stack software:

- spreco di risorse
 - conflitti tra utenti
 - impossibilità di garantire prestazioni e equità
-

Visione a strati dello stack HPC

Lo **stack HPC** è organizzato in strati, ciascuno con un ruolo specifico

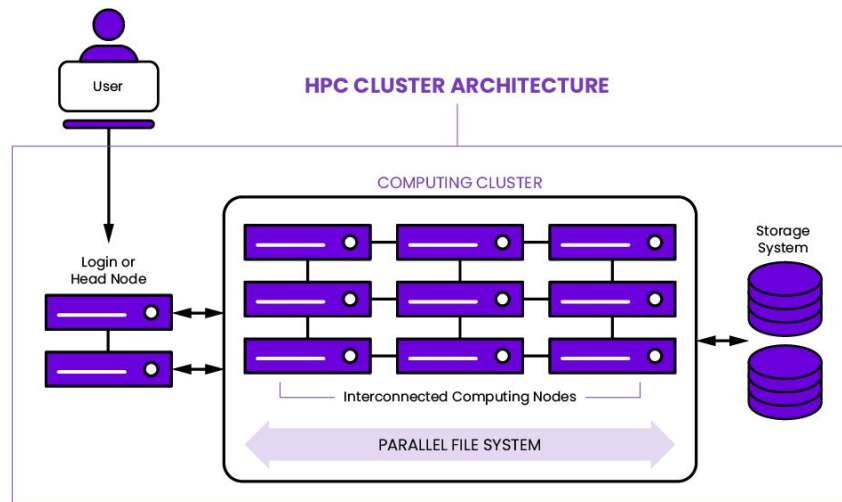


Ogni livello interagisce con quelli adiacenti e influenza il comportamento prestazionale del sistema

Cos'è un sistema HPC (vista utente)

Dal punto di vista dell'utente, un sistema HPC è:

- un **ambiente di calcolo controllato**, non una singola macchina
- l'integrazione di hardware , software e policy di utilizzo
- un'infrastruttura pensata per:
 - eseguire job di lunga durata
 - massimizzare l'utilizzo complessivo delle risorse
- un sistema in cui:
 - le risorse sono condivise e non sempre disponibili
 - l'accesso al calcolo è mediato



Login node e compute node

Login Node



- punto di accesso per gli utenti
- utilizzo interattivo
- attività consentite:
 - a. editing del codice
 - b. compilazione*
 - c. preparazione dei job
- non destinato a calcoli intensivi

scheduler

Compute Node



- risorse dedicate al calcolo
- accesso mediato dallo scheduler
- esecuzione dei job batch
- utilizzabili solo con un job assegnato dallo scheduler

Perché questa separazione esiste

La separazione tra login node e compute node serve a:

- Garantire la **stabilità** del sistema
- Proteggere le **risorse di calcolo**
- Evitare interferenze tra utenti
- Assicurare un utilizzo **equo ed efficiente**
- Permettere una pianificazione controllata dei job

hardware costoso → va gestito e protetto

Filesystem nei sistemi HPC

Caratteristiche tipiche dei filesystem HPC:

- **Filesystem condiviso** tra login node e compute node
 - Accessibile contemporaneamente da molti nodi
 - Progettato per:
 - elevato **throughput**
 - accessi paralleli
 - Spesso implementato come:
 - filesystem parallelo
 - Directory tipiche:
 - \$HOME → dati personali, configurazioni
 - \$SCRATCH → dati temporanei, output dei job
-

Implicazioni del filesystem

L'I/O nei sistemi HPC ha impatto diretto sulle prestazioni e sulla scalabilità.

- **Molti file piccoli:**
 - elevato overhead
 - degradazione delle prestazioni globali
- **Accessi casuali:**
 - penalizzano throughput e latenza
 - Non sfruttano la struttura del FS parallelo
- **I/O parallelo:**
 - necessario per grandi volumi di dati
 - Accesso coordinato da più processi

L'I/O è spesso il **vero limite alla scalabilità** anche quando il calcolo scala correttamente.

Esempio reale (Cineca)

https://docs.hpc.cineca.it/hpc/hpc_data_storage.html

- \$HOME ha backup ed è pensato per programmi e piccoli dati, \$WORK e \$SCRATCH **non hanno backup.**
 - \$SCRATCH è **temporaneo**: CINECA dice che i file non acceduti da più di **40 giorni** vengono cancellati automaticamente.
 - Lustre è ottimo per **grandi file e accessi paralleli**, ma soffre con **molti file piccoli**
-

Compilatori in ambiente HPC

Nei sistemi HPC sono tipicamente disponibili più compilatori:

- **GCC**
- **Clang/LLVM**
- Compilatori vendor-specific (es. Intel, AOCC, NVIDIA HPC)

Ruolo del compilatore:

- traduce il codice sorgente
- applica ottimizzazioni automatiche
- genera codice ottimizzato per l'architettura

il compilatore è **parte integrante delle prestazioni**

Perchè più compilatori

Nei sistemi HPC coesistono più compilatori perché:

- Le **architetture hardware** sono diverse
- Ogni compilatore implementa:
 - strategie di ottimizzazione differenti
 - modelli diversi di vettorizzazione e parallelismo
- Alcune librerie sono:
 - ottimizzate per compilatori specifici
- Portabilità e prestazioni sono spesso in **tensione**

non esiste un compilatore “migliore” in assoluto

Librerie fondamentali

Nei sistemi HPC le librerie sono componenti centrali dello stack software

- Forniscono **implementazioni ottimizzate** di operazioni fondamentali
- Incapsulano:
 - conoscenza dell'architettura
 - ottimizzazioni avanzate
- Esempi di librerie fondamentali:
 - **MPI** – comunicazione tra processi
 - **OpenMP** – parallelismo a memoria condivisa
 - **BLAS / LAPACK** – algebra lineare
- Spesso le librerie:
 - determinano più delle prestazioni
 - del codice applicativo stesso

<https://www.open-mpi.org/>

<https://www.openmp.org/>

<https://www.netlib.org/blas/>

<https://www.netlib.org/lapack/>

Il problema dell'ambiente software

Nei sistemi HPC l'ambiente software è intrinsecamente complesso rispetto ad un sistema classico.

- Coesistono:
 - molte versioni di compilatori
 - molte versioni di librerie
- Le applicazioni richiedono:
 - dipendenze specifiche di librerie
 - requisiti spesso incompatibili
- L'ambiente deve essere:
 - controllato
 - riproducibile
 - configurabile per utente e per job

MPI compilato con GCC $\neq$ MPI compilato con Intel

Environment modules

Gli **Environment Modules** sono il meccanismo standard per gestire l'ambiente software in HPC:

- Permettono di:
 - caricare versioni specifiche di software
 - configurare l'ambiente per utente e per job
 - Agiscono modificando:
 - variabili d'ambiente: PATH, LD_LIBRARY_PATH, ecc.
 - Consentono la coesistenza di:
 - più compilatori
 - più librerie
 - più versioni dello stesso software
 - Favoriscono:
 - controllo
 - flessibilità
 - riproducibilità
-

Esempio reale (Cineca)

https://docs.hpc.cineca.it/hpc/hpc_enviroment.html

Runtime environment

Il **runtime environment** è il livello software che gestisce l'esecuzione del programma parallelo a tempo di run-time.

Non fa parte del kernel del sistema operativo, ma opera in user-space sopra di esso.

Cosa fa:

- Crea e gestisce thread (OpenMP)
- Inizializza e coordina processi (MPI)
- Gestisce sincronizzazione (barrier, lock, collectives)
- Coordina la comunicazione tra processi
- Applica CPU binding e memory affinity
- Implementa politiche di scheduling interno

È influenzato da

- Variabili d'ambiente (OMP_NUM_THREADS, OMP_PROC_BIND, ecc.)
 - Parametri del job (SLURM: task, core, nodi)
 - Architettura del nodo (NUMA, SMT)
-

Perchè serve uno scheduler

- Nei sistemi HPC lo scheduler è indispensabile perché:
 - Le risorse sono:
 - limitate, condivise, costose
 - Molti utenti effettuano richieste:
 - diverse, concorrenti spesso incompatibili
 - È necessario:
 - pianificare l'uso delle risorse
 - evitare conflitti
 - garantire equità
 - massimizzare l'efficienza complessiva
 - Senza scheduler:
 - caos operativo
 - spreco di risorse
 - instabilità del sistema
-

Cosa fa uno scheduler

Uno **scheduler** nei sistemi HPC:

- Riceve le **richieste di risorse** dagli utenti (job)
- Decide:
 - **quando** un job può partire
 - **dove** (su quali nodi)
- Assegna e isola le risorse:
 - CPU / core
 - memoria
 - eventualmente GPU
- Applica **policy di sistema**:
 - priorità
 - limiti di tempo
 - fair-share

Slurm è lo scheduler standard di fatto

<https://slurm.schedmd.com/>

Job batch: concetto

Nei sistemi HPC l'unità fondamentale di esecuzione è il **job**:

- Un **job** describe:
 - quali risorse sono necessarie
 - per quanto tempo
 - quale programma eseguire
- Il job viene:
 - sottomesso allo scheduler
 - messo in coda
 - eseguito quando le risorse sono disponibili
- Il job è:
 - un'entità **schedulata**
 - diversa da un processo tradizionale

In HPC non si esegue direttamente un programma, si richiedono risorse

workflow tipico HPC

Il workflow tipico di utilizzo di un sistema HPC è il seguente

1. **Accesso al sistema**
 - login sul login node
2. **Preparazione dell'ambiente**
 - caricamento dei moduli software
3. **Sviluppo e compilazione**
 - scrittura del codice
 - compilazione con il compilatore scelto
4. **Preparazione del job**
 - definizione delle risorse richieste
 - creazione dello script di job
5. **Sottomissione del job**
 - invio allo scheduler
6. **Esecuzione**
 - il job gira sui compute node
7. **Analisi dei risultati**
 - output
 - log
 - eventuale debugging

Questo ciclo viene ripetuto più volte durante lo sviluppo

Errori tipici

Errori comuni quando si inizia ad utilizzare sistemi HPC:

- Eseguire calcoli intensivi sul **login node** ignorando lo **scheduler**
 - Richiedere risorse non adeguate:
 - troppe (spreco)
 - troppo poche (job inefficiente)
 - Usare il filesystem sbagliato:
 - \$HOME per grandi output
 - Pensare all'HPC come a un PC potente
 - Non considerare:
 - ambiente software
 - runtime
 - policy del sistema
-

Collegamento al laboratorio

Nel laboratorio applicheremo i concetti visti:

- Accesso a un sistema HPC
- Uso del **login node**
- Caricamento dei **moduli software**
- Compilazione di un semplice codice
- Preparazione e sottomissione di un **job**
- Interazione con lo **scheduler**
- Analisi dell'output e dei log

Utilizzeremo il seguente cluster slurm (docker)

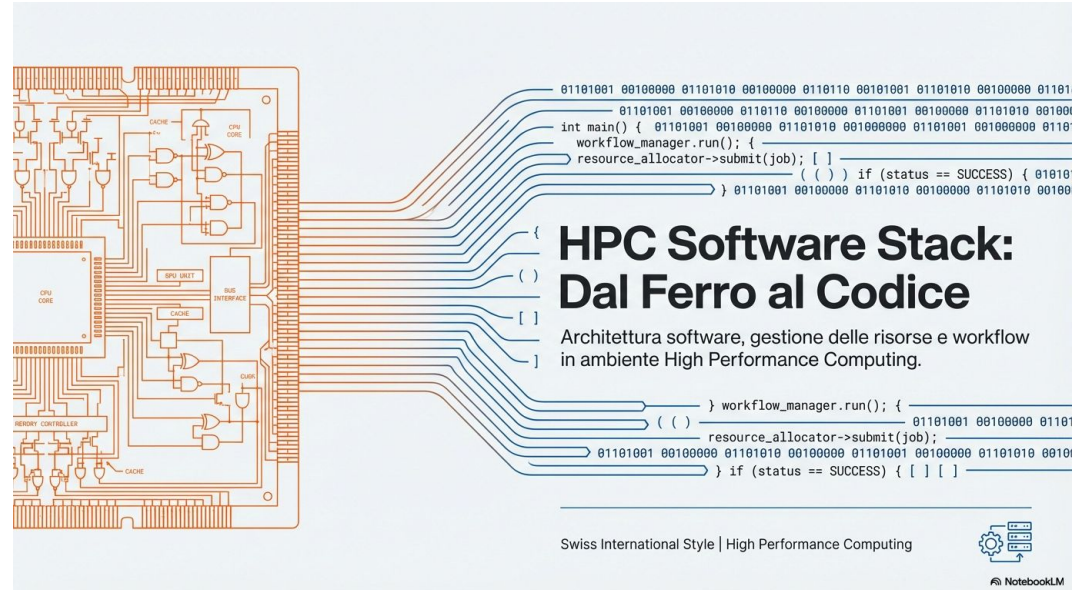
<https://github.com/fvitello/slurm-docker-cluster>

Podcast

Questo podcast è stato generato con strumenti di Intelligenza Artificiale (<https://notebooklm.google/>) utilizzando contenuti e fonti forniti dall'autore del corso.

Il materiale ha esclusivamente funzione di supporto e sintesi degli argomenti trattati a lezione. Non sostituisce il materiale didattico ufficiale, le slide, i testi di riferimento o lo studio individuale.

Il podcast deve essere inteso come uno strumento integrativo per il ripasso e il consolidamento dei concetti, e non come fonte esaustiva o alternativa allo studio approfondito.



<https://youtu.be/1nOCpyYHOsM>